

objects, `ballList`. Each `Ball` object chooses and remembers a randomized starting location and a randomized speed in both the x and y directions.

Inside the main loop, we iterate through all the `Ball` objects and tell each one to update itself ❷, changing the x- and y-coordinates of each `Ball` object to a new location. We then iterate through the list again, calling the `draw()` method of each `Ball` object ❸.

When we run the program, we see three balls, each starting at a randomized location and each moving with a randomized x and y speed. Each ball bounces correctly off the boundaries of the window.

Using this object-oriented approach, we made no changes to the `Ball` class, but just changed our main program to now manage a list of `Ball` objects instead of a single `Ball` object. This is a common, and very positive, side effect of OOP code: well-written classes can often be reused without change.

Creating Many, Many Ball Objects

We can change the value of the constant `N_BALLS` from 3 to some much larger value, like 300, to quickly create that many balls ([Figure 6-1](#)). By changing just a single constant, we make a major change to the behavior of the program. Each ball maintains its own speed and location and draws itself.